CAPSTONE PROJECT 03

PolicyPal: Enterprise HR and IT Policy Chatbot

Sakura Logistics, 4,000 employees in India and Japan, with HR and IT policies spread across 120 documents.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

1. The customer's problem

HR and IT help desks receive hundreds of repetitive questions a week. Employees cannot find answers in long PDFs and sometimes act on outdated information. The company wants an assistant that answers from current policies with citations, refuses when unsure, protects personal data, and hands over to a human with context.

2. Users

- Employees
- HR and IT help desk agents
- Policy owners who maintain documents
- Security and compliance team

3. User stories

- As an employee, I ask "Can I carry forward my leave?" and get a short answer with the policy section cited.
- As an employee, when the assistant cannot answer, I am handed to a human with my question summarised.
- As a policy owner, I upload an updated policy and answers change the same day.
- As security, I see that personal data is redacted and injection attempts are blocked.
- As the help desk lead, I see the most common unanswered topics each week.

4. Requirements

Must have

- Ingestion pipeline for Markdown and PDF policies with structure-aware chunking and metadata (Day 22)
- Hybrid retrieval with re-ranking and corrective RAG (Days 23, 26)
- Grounded answers with citations; refusal below a relevance threshold
- Guardrails: injection scanning on documents and inputs, PII redaction on outputs (Day 28)
- Human handoff creating a ticket with a structured payload (Day 27)

- Golden set of 30 items and a CI quality gate (Days 29, 30)
- React chat UI with citations and feedback buttons

Should have

- Re-ingestion on document change with index versioning
- Weekly report of unanswered topics and drift (Day 35)
- Japanese questions supported and evaluated

Could have (stretch)

- Role-based access to confidential policies
- Conversation memory with summaries
- Teams or Slack integration design

5. Data

Starter notebook writes 8 sample policy documents (leave, remote work, expenses, travel, security, IT, code of conduct, visa) and 40 example employee questions.

6. Suggested AI approach

Context engineering and chunking (Day 22), hybrid search and re-ranking (Day 23), CRAG (Day 26), guardrails (Day 28), evaluation and gates (Days 29, 30).

7. Suggested architecture

```
React UI -> FastAPI back end -> database (SQL / MongoDB)
              |-> AI service (retrieval, model, tools, guardrails)
              |-> logs and metrics -> dashboard / weekly report
CI/CD: tests + quality gate -> Docker images
       -> Azure (Container Apps or App Service)
```

8. How to do it: day by day

Day 36 (Mon Dec 14): Kick-off and design

- Quiz on Day 35
- Teams of 3 to 4 formed; project chosen
- Problem statement, users, success metrics, assumptions log
- Architecture diagram and data model

Day 37 (Tue Dec 15): Core build

- Stand-up (15 min)
- Data loaded; back end API with main endpoints
- AI core working end to end (retrieval, recommendation, agent or pipeline)
- Golden set draft of 20 items

Day 38 (Wed Dec 16): Complete features and quality

- Stand-up
- Front end connected to the API
- Guardrails and error handling
- Evaluation run and first error analysis

Day 39 (Thu Dec 17): Harden, deploy, observe

- Stand-up
- Docker images; deploy to Azure or a documented local deployment
- Structured logs and a monitoring view
- Red-team and slice checks; fix top issues

Day 40 (Fri Dec 18): Demo day

- Final rehearsal (morning)
- Demo: 10 minutes + 5 minutes Q&A; per team
- Evaluator scoring with the rubric
- Individual reflection and peer feedback

9. Project-specific acceptance checks

Evaluators run these during or after the demo. They count towards Core functionality and AI quality.

- Faithfulness on the golden set at least 0.9
- Unanswerable questions refused
- Injection test suite: block rate at least 90%, false alarms at most 10%
- Citations point to the right section

10. Golden set: starter items

Extend these to at least 20 items (30 for PolicyPal) with expected answers, sources and tags.

- "When does carried forward leave expire?" must include 31 March
- "Can I use any AI tool with customer data?" must say only approved tools
- "Is there a company gym?" must refuse politely

11. How you will be evaluated

Criterion	Points	For this project, evaluators look for
Problem understanding and scoping	10	Clear problem statement, users, success metrics, assumptions log, agreed MVP scope (Days 15, 20).
Architecture and design	10	Sensible architecture diagram, layered code, right tool choices with reasons, data model (Days 4, 6, 7, 22, 33).
Core functionality	20	All must-have features work end to end in the demo; project checklist items delivered.
AI and data quality	15	Golden set of at least 20 items, measured metrics (retrieval, faithfulness or recommendation quality), error analysis (Days 23, 29, 30).
Engineering quality	10	Readable code, tests (API, pipeline, prompts), Git history with pull requests, CI pipeline green (Days 13, 17, 18).
Security, guardrails and responsible AI	10	Input and output guardrails, secrets handling, least privilege, a red-team check, fairness or language slice check (Days 8, 28).
Deployment and operations	10	Containerised, deployable (Azure or documented), structured logs, a dashboard or report, one drift or quality check (Days 19, 31 to 35).
Demo and communication	10	Answer-first readout, live demo, honest limits, confident Q&A;, clear handover README (Days 15, 20, 24).
Teamwork and process	5	Daily stand-ups, decision log, fair contribution, kind and specific feedback (Days 15, 24, 31).

12. Common pitfalls

- No refusal path, so the bot invents policies
- Secrets or personal data in logs
- Testing only with questions copied from the documents

13. Getting started

- Open Starter_Notebooks/Capstone_03_PolicyPal_Starter.ipynb in Colab or VS Code to generate the data and see a working baseline.
- Use App_Build_Prompts/Capstone_03_PolicyPal_Prompts.md with your AI coding assistant to scaffold the project.
- Write your problem statement and assumptions log before writing code.

Ganbatte (頑張って): do your best